

Notebook Algoritmos ICPC Python

Contents

1	Entrada/Salida Rápida en Python	2	7	Construcción y Combinatoria	13
1.1	Lectura línea a línea (caso general) .	2	7.1	Algoritmos Constructivos: Invariantes y Paridad	13
1.2	Lectura total + tokenización (máxima velocidad)	2	7.2	Algoritmos Constructivos: Construcción Explícita	14
1.3	Casos con centinela / bandera	2	7.3	Combinatoria: Conteo por Simetría de Binomios	14
1.4	Casos hasta EOF (sin contador T) . .	2	7.4	+Verificación de Factibilidad por Invariante de Clases——	15
1.5	Salida rápida	2			
2	Herramientas y Funciones Esenciales de Python	2			
3	Estructuras de Datos	3			
3.1	Fenwick Tree (Árbol de Índices Binarios / BIT)	3			
4	Algoritmos de Grafos	4			
4.1	Emparejamiento Bipartito Máximo (Algoritmo de Kuhn)	4			
4.2	Cobertura Mínima de Vértices en Grafos Bipartitos (Teorema de König)	5			
4.3	Emparejamiento Bipartito por Segmentación de Filas/Columnas	5			
4.4	Componentes Fuertemente Conexas (Algoritmo de Kosaraju)	6			
4.5	Circuito Euleriano (Algoritmo de Hierholzer)	7			
4.6	Puntos de Articulación (Algoritmo de Tarjan)	8			
5	Búsqueda y Optimización	9			
5.1	Búsqueda Binaria sobre la Respuesta + Two Pointers	9			
5.2	——NEED: Enumeración por Máscaras de Bits (Bitmask Brute Force) .	9			
6	Matemáticas y Simulación	10			
6.1	Patrón de Reducción Binaria (tipo Josefo)	10			
6.2	Fuerza Bruta / Simulación Directa .	10			
6.3	Simulación Lineal (Máximo de Prefijo)	11			
6.4	+Ordenamiento de Puntos Críticos y Búsqueda de Huecos——	11			
6.5	+Teoría de Números: Regla de Divisibilidad Alternada——	12			
6.6	+Simulación de Eventos con Colas——	12			

Cada entrada de este cuaderno sigue siempre la misma estructura: preguntas guía para reconocer cuándo aplica el algoritmo en el contrarreloj, para qué sirve, su complejidad, el fundamento teórico, una plantilla genérica en Python comentada, y un ejercicio real resuelto con esa técnica. Este es un documento vivo: se irá ampliando problema a problema a medida que se resuelvan más ejercicios de cara a la Clasificatoria Nacional ICPC Colombia 2026.

1 Entrada/Salida Rápida en Python

Antes de entrar en los algoritmos: en Python la I/O es el cuello de botella más común en ICPC. Estas son las 4 situaciones típicas y el patrón recomendado para cada una.

1.1 Lectura línea a línea (caso general)

Cuando cada línea tiene una cantidad de datos conocida y el número de líneas no es extremo (hasta 10^5). Es el patrón por defecto.

```
import sys
input = sys.stdin.buffer.readline
n = int(input())
arr = list(map(int, input().split()))
```

1.2 Lectura total + tokenización (máxima velocidad)

Cuando hay muchísimos números sueltos (10^5 – 10^6 +) sin importar en cuántas líneas están repartidos. Es la forma más rápida en Python: una sola llamada al sistema.

```
import sys
data = sys.stdin.buffer.read().split()
idx = 0
n = int(data[idx]); idx += 1
arr = list(map(int, data[idx:idx + n])); idx += n
```

1.3 Casos con centinela / bandera

El input termina cuando aparece un valor especial (p. ej. "0 0"). No se sabe de antemano cuántos casos hay.

```
import sys
input = sys.stdin.buffer.readline
while True:
    a, b = map(int, input().split())
    if a == 0 and b == 0: # centinela de fin
        break
    # ... procesar caso ...
```

1.4 Casos hasta EOF (sin contador T)

No hay número de casos declarado; se procesa hasta que el flujo se acaba.

```
import sys
for line in sys.stdin:
    line = line.strip()
    if not line:
```

```
        continue
    # ... procesar línea ...
# alternativa igual de rápida:
# data = sys.stdin.buffer.read().split()
# idx = 0
# while idx < len(data): ...
```

1.5 Salida rápida

Nunca usar `print()` dentro de un loop grande: acumula resultados en una lista y escribe todo de una sola vez.

```
ans = []
# ans.append(resultado) dentro del loop
sys.stdout.write("\n".join(map(str, ans)) + "\n")
```

2 Herramientas y Funciones Esenciales de Python

Funciones y módulos de la librería estándar que aparecen todo el tiempo en soluciones de ICPC. No son algoritmos en sí, pero escribir código correcto y rápido depende de conocerlas bien.

`enumerate()`, `zip()`, `iter()`, `any()` / `all()`, `reversed()`

`enumerate(it, start=0)` recorre con índice sin llevar un contador aparte; `zip(a, b, ...)` recorre varias listas en paralelo y se detiene en la más corta; `iter(x)` crea un iterador manual útil para consumir tokens uno a uno; `any()/all()` verifican una condición sobre un iterable, cortando en cuanto se decide el resultado; `reversed(seq)` recorre de atrás hacia adelante sin copiar la secuencia.

```
for i, x in enumerate(arr): # i arranca en 0
    ...
for i, x in enumerate(arr, 1): # i arranca en 1 (util
    en 1-indexado)
    ...

for x, y in zip(a, b): # recorre a y b en
    paralelo
    ...
pares = list(zip(a, b)) # combina en tuplas (a
    [i], b[i])

it = iter(data) # consumir tokens sin
    indice manual
n = int(next(it))
arr = [int(next(it)) for _ in range(n)]

if any(x % 2 == 0 for x in arr): # hay al menos un par
    ...
if all(x > 0 for x in arr): # todos son positivos
    ...

for x in reversed(arr): # recorre de atras
    hacia adelante
    ... # (mas barato que arr
   [::-1], no copia)
```

Expresiones regulares (módulo `re`)

Útiles para extraer números o tokens de strings con formato irregular, validar patrones, o separar por delimitadores complejos. Funciones clave: `re.match` (busca solo al inicio), `re.search` (busca en cualquier parte, primer match), `re.findall` (todas las coincidencias como lista),

re.sub (reemplaza) y re.split (separa usando el patrón como delimitador).

Patrones comunes: \d dígito, \D no-dígito, \w alfanumérico + guion bajo, \s espacio en blanco; + una o más, * cero o más, ? cero o una, {n,m} entre n y m repeticiones; [...] conjunto de caracteres, [...] negación; ^ inicio y \$ fin de string; (...) grupo de captura.

```
import re

nums = re.findall(r'[-?\d+]', s)      # todos los enteros
                                     # (con signo) del string
nums = list(map(int, nums))

tokens = re.split(r'\s+', s.strip()) # separa por
                                     # cualquier cantidad de espacios

m = re.match(r'^(\d+)-(\d+)$', s)    # válida "3-5" y
                                     # captura ambos numeros
if m:
    a, b = int(m.group(1)), int(m.group(2))

limpio = re.sub(r'[^a-zA-Z0-9]', '', s) # deja solo
                                     # alfanumericos
```

dict y defaultdict

defaultdict evita chequear si una clave existe antes de usarla: crea automáticamente un valor por defecto (int → 0, list → [], set → vacío) la primera vez que se accede a una clave nueva.

```
from collections import defaultdict

freq = {}
freq[x] = freq.get(x, 0) + 1        # patron clasico sin
                                     # defaultdict

freq = defaultdict(int)              # no hace falta
                                     # chequear si la clave existe
freq[x] += 1

grupos = defaultdict(list)          # agrupar elementos
                                     # por clave
for x in arr:
    grupos[x % 3].append(x)

freq.get(x, valor_default)           # no lanza KeyError
                                     # si x no existe
x in freq                            # O(1) para chequear
                                     # existencia
sorted(freq.items(), key=lambda kv: -kv[1]) # ordenar
                                     # por frecuencia desc
```

replace() y split()

```
s.replace("a", "b")                  # reemplaza TODAS las
                                     # ocurrencias de "a" por "b"
s.replace("a", "b", 1)               # reemplaza solo la primera
                                     # ocurrencia

s.split()                            # separa por espacios en
                                     # blanco (ignora vacios)
s.split(",")                          # separa por delimitador
                                     # exacto (puede dar strings vacios)
s.strip()                            # quita espacios/saltos de
                                     # linea al inicio y final
s.split("\n")                         # separar por lineas
                                     # manualmente
```

Precisión de números y formatos

round() puede fallar por representación binaria de decimales; para imprimir con una cantidad exacta de decimales es más seguro formatear directamente. Cuando el punto flotante no alcanza (aritmética exacta con fracciones), usar el módulo fractions.

```
round(3.14159, 2)                    # 3.14 (cuidado: no siempre es
                                     # exacto)
f"{x:.2f}"                           # formatea con exactamente 2
                                     # decimales
f"{x:.10f}"                           # 10 decimales, util si el
                                     # juez pide tolerancia 1e-9

from fractions import Fraction        # aritmetica EXACTA con
                                     # fracciones
Fraction(1, 3) + Fraction(1, 6)      # -> Fraction(1, 2),
                                     # sin error de redondeo

x // y                                # division entera, redondea
                                     # hacia -infinito
-7 // 2                               # -4 (no -3): distinto a otros
                                     # lenguajes
```

sys.setrecursionlimit(n)

```
import sys
sys.setrecursionlimit(200000)        # el limite por defecto
                                     # (~1000) es muy bajo
                                     # # para DFS recursivo en
                                     # grafos/arboles grandes

# aun asi, Python puede hacer stack overflow real (crash)
# con recursion muy
# profunda (~10^4-10^5), sin importar el limite
# configurado. Para n grande,
# preferir SIEMPRE DFS iterativo con pila explicita (ver
# seccion II, SCC).
```

3 Estructuras de Datos

3.1 Fenwick Tree (Árbol de Índices Binarios / BIT)

Preguntas guía — ¿cuándo usarlo?

- ¿Necesito sumas o conteos acumulados sobre un rango que cambia con actualizaciones puntuales?
- ¿Los updates son de un solo índice y las queries son sobre prefijos o rangos [l, r]?
- ¿Si no hay updates "online", puedo resolver offline ordenando las queries (p. ej. por su extremo derecho)?

¿Para qué sirve? Mantener sumas de prefijo (o cualquier operación invertible) sobre un arreglo con actualizaciones puntuales, respondiendo consultas de rango en tiempo logarítmico. Es la base de técnicas como conteo de elementos distintos por rango, inversiones en un arreglo y frecuencias acumuladas.

Complejidad: Construcción $O(n)$. Update puntual $O(\log n)$. Query de prefijo $O(\log n)$. Espacio $O(n)$.

Fundamento teórico: Cada posición i del arreglo interno almacena la suma de un bloque de tamaño $(i \& -i)$, el bit menos significativo de i . Al actualizar se "sube" sumando $(i \& -i)$; al consultar un prefijo se "baja" restando $(i \& -i)$. Esto aprovecha la representación binaria de los índices para cubrir cualquier prefijo con solo $O(\log n)$ bloques.

Ejercicio aplicado

Dado un arreglo de n números y q consultas (i, j) , responder para cada una cuántos valores distintos hay en el subarreglo $[i, j]$. Técnica: procesar las consultas offline ordenadas por extremo derecho r , recorriendo el arreglo de izquierda a derecha y manteniendo en el BIT un "1" solo en la última aparición de cada valor.

E/S: $n \leq 30\,000$ y $q \leq 200\,000 \rightarrow$ hasta $\sim 430\,000$ tokens de entrada: se usa lectura total + tokenización (patrón 2) en vez de readline por línea.

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.readline
    n = int(data())
    nums = list(map(int, data().split()))
    q = int(data())

    queries = []
    for i in range(q):
        l, r = map(int, data().split())
        queries.append((r, l, i))      # se ordena por r
    (offline)
    queries.sort()

    bit = [0] * (n + 1)

    def update(i, val):
        while i <= n:
            bit[i] += val
            i += i & (-i)      # ultimo bit encendido

    def query(i):
        res = 0
        while i > 0:
            res += bit[i]
            i -= i & (-i)
        return res

    last = {}                  # ultima posicion
    # vista de cada valor
    ans = [0] * q
    cur = 0
    for r, l, qi in queries:
        while cur < r:         # avanza el
            puntero hasta r
            cur += 1
            x = nums[cur - 1]
            prev = last.get(x)
            if prev:
                update(prev, -1)    # desactiva la
            # aparicion anterior
            update(cur, 1)          # activa la
            # aparicion actual
            last[x] = cur
            ans[qi] = query(r) - query(l - 1)

    sys.stdout.write("\n".join(map(str, ans)))

solve()
```

4 Algoritmos de Grafos

4.1 Emparejamiento Bipartito Máximo (Algoritmo de Kuhn)

Preguntas guía — ¿cuándo usarlo?

- ¿Tengo dos conjuntos disjuntos (personas / tareas) y aristas que solo conectan un conjunto con el otro?
- ¿Me piden el número máximo de "parejas" o asignaciones simultáneas sin repetir elementos de ningún lado?
- ¿El tamaño de ambos lados permite $O(V \cdot E)$ (hasta unos pocos miles)?

¿Para qué sirve? Encontrar el máximo número de aristas seleccionables en un grafo bipartito sin que dos aristas compartan vértice (asignación óptima uno a uno). Ejemplos: asignar trabajadores a tareas, parejas de baile, cobertura de tareas.

Complejidad: $O(V \cdot E)$ con la versión simple de caminos aumentantes, donde V es el lado izquierdo y E el número de aristas.

Fundamento teórico: Se buscan repetidamente "caminos aumentantes": para cada vértice izquierdo se intenta, vía DFS, encontrar una pareja libre o "desalojar" a otro vértice ya emparejado para liberar espacio y aumentar el emparejamiento en 1. Es un caso particular del teorema de König / flujo máximo en redes.

Ejercicio aplicado

Hay n chicos y m chicas y k parejas de baile posibles. Hallar el número máximo de parejas simultáneas y mostrar una asignación válida.

E/S: $n, m \leq 500$ y $k \leq 1000 \rightarrow$ entrada pequeña, se usa readline por línea (patrón 1), suficiente y más simple de leer.

Código solución

```
import sys
input = sys.stdin.buffer.readline

def try_kuhn(u, adj, matched, visited):
    # intenta emparejar el vertice u del lado izquierdo
    for v in adj[u]:
        if not visited[v]:
            visited[v] = True
            # v esta libre, o su pareja actual puede reubicarse...
            if matched[v] == -1 or try_kuhn(matched[v],
                adj, matched, visited):
                matched[v] = u
                return True
    return False

def solve():
    n, m, k = map(int, input().split())
    adj = [[] for _ in range(n)]
    for _ in range(k):
        a, b = map(int, input().split())
        adj[a - 1].append(b - 1)

    matched = [-1] * m        # pareja actual
    # de cada vertice derecho
    total = 0
    for u in range(n):
        visited = [False] * m    # se reinicia en
        # cada intento
        if try_kuhn(u, adj, matched, visited):
            total += 1

    out = [str(total)]
    for v in range(m):
        if matched[v] != -1:
            out.append(f"{matched[v] + 1} {v + 1}")
```

```
sys.stdout.write("\n".join(out))

solve()
```

4.2 Cobertura Mínima de Vértices en Grafos Bipartitos (Teorema de König)

Preguntas guía — ¿cuándo usarlo?

- ¿Piden el mínimo número de elementos a eliminar para que ningún conflicto (arista) quede activo?
- ¿El grafo de conflictos es bipartito (se puede partir en 2 grupos donde todo conflicto va de un grupo al otro)?
- ¿Ya sé calcular el emparejamiento máximo de ese grafo (entrada anterior, Kuhn)?

¿Para qué sirve? Calcular la cobertura mínima de vértices (mínimo conjunto de nodos que toca todas las aristas) en un grafo bipartito. En grafos generales este problema es NP-difícil, pero en grafos bipartitos se vuelve polinomial gracias al teorema de König.

Complejidad: $O(V \cdot E)$, la misma del emparejamiento bipartito máximo del que depende.

Fundamento teórico: El teorema de König establece que, en cualquier grafo bipartito, el tamaño de la cobertura mínima de vértices es EXACTAMENTE igual al tamaño del emparejamiento máximo. Basta con correr Kuhn (entrada anterior) y esa cifra ya es la respuesta. Importante: heurísticas locales (como quitar iterativamente puntos de articulación, o dividir el número de vértices con grado > 0 entre 2) NO son válidas para este problema en general — solo coinciden por casualidad en casos muy simples (aristas aisladas, caminos, estrellas) y fallan en cuanto el grafo tiene una estructura más rica (por ejemplo, un grafo bipartito 2-conexo sin puntos de articulación).

Ejercicio aplicado

Dado un tablero $N \times N$ con K caballos de ajedrez colocados, hallar el mínimo número de caballos que hay que quitar para que ninguno ataque a otro. El grafo de ataques es bipartito por construcción: cada movimiento de caballo cambia la paridad de (fila + columna), así que alcanza con particionar por esa paridad y correr Kuhn.

E/S: $N \leq 25$, $K \leq N^2 \leq 625 \rightarrow$ entrada pequeña, readline por línea (patrón 1).

Código solución

```
import sys
input = sys.stdin.buffer.readline

def try_kuhn(u, adj, matched, visited):
    for v in adj[u]:
        if not visited[v]:
            visited[v] = True
            if matched[v] == -1 or try_kuhn(matched[v],
            adj, matched, visited):
                matched[v] = u
                return True
    return False

def max_bipartite_matching(left_nodes, n_right, adj):
    matched = [-1] * n_right
```

```
total = 0
for u in left_nodes:
    visited = [False] * n_right
    if try_kuhn(u, adj, matched, visited):
        total += 1
return total

def solve():
    n, k = map(int, input().split())
    pos = []
    for _ in range(k):
        r, c = map(int, input().split())
        pos.append((r - 1, c - 1))

    index = {p: i for i, p in enumerate(pos)}
    moves = [(1,2),(2,1),(-1,2),(-2,1),(1,-2),(2,-1),
    (-1,-2),(-2,-1)]

    # el ataque de caballo SIEMPRE cambia el color (fila+
    # col) par/impar:
    # el grafo de ataques es bipartito por construccion,
    # sin necesidad
    # de demostrarlo -- solo hay que particionar por ese
    # color
    left_nodes = [i for i, (r, c) in enumerate(pos) if (r
    + c) % 2 == 0]
    adj = [[] for _ in range(k)]
    for i in left_nodes:
        r, c = pos[i]
        for dr, dc in moves:
            j = index.get((r + dr, c + dc))
            if j is not None:
                adj[i].append(j)

    # Konig: cobertura minima de vertices ==
    # emparejamiento maximo
    print(max_bipartite_matching(left_nodes, k, adj))

solve()
```

4.3 Emparejamiento Bipartito por Segmentación de Filas/Columnas

Preguntas guía — ¿cuándo usarlo?

- ¿La cuadrícula tiene obstáculos que "cortan" el alcance de piezas o rangos dentro de cada fila y columna?
- ¿Cada fila y cada columna se puede dividir en segmentos independientes (separados por obstáculos) donde a lo más un elemento puede colocarse por segmento?
- ¿Piden el máximo número de elementos colocables sin conflicto entre sí?

¿Para qué sirve? Resolver problemas de colocación de piezas sobre una cuadrícula con obstáculos, donde el conflicto es "compartir fila o columna sin un obstáculo de por medio". En vez de razonar celda por celda, se etiquetan los segmentos libres de cada fila y columna, y el problema se convierte en un emparejamiento bipartito clásico.

Complejidad: $O(N^2)$ para etiquetar segmentos, más $O(V \cdot E)$ para el emparejamiento (V , E acotados por N^2).

Fundamento teórico: Cada casilla vacía pertenece a exactamente un segmento de fila (corrida maximal de casillas vacías, delimitada por obstáculos o bordes) y a exactamente un segmento de columna. Como a lo más una torre puede ocupar cada segmento (todas sus casillas se

atacan entre sí sin obstáculo de por medio), colocar torres sin conflicto equivale a elegir aristas entre segmentos de fila y segmentos de columna sin repetir segmento de ningún lado: exactamente un emparejamiento en el grafo bipartito (segmentos de fila) $\times$ (segmentos de columna), donde cada casilla vacía es una arista posible.

Ejercicio aplicado

Dado un tablero $N \times N$ con algunas casillas ocupadas por peones, hallar el máximo número de torres que se pueden colocar en casillas vacías sin que se ataquen entre sí (dos torres se atacan si comparten fila o columna sin un peón entre ellas). La entrada trae varios tableros seguidos.

E/S: $N \leq 100$, múltiples casos hasta EOF (patrón 4 de la sección de E/S) $\rightarrow$ lectura total con `sys.stdin.read().split()` y procesamiento por índice.

Código solución

```
import sys

def try_kuhn(u, adj, matched, visited):
    for v in adj[u]:
        if not visited[v]:
            visited[v] = True
            if matched[v] == -1 or try_kuhn(matched[v],
            adj, matched, visited):
                matched[v] = u
                return True
    return False

def solve_board(n, grid):
    # etiqueta cada corrida maximal de celdas vacias en
    # cada FILA con un id
    row_id = [[-1] * n for _ in range(n)]
    row_count = 0
    for r in range(n):
        c = 0
        while c < n:
            if grid[r][c] == '.':
                while c < n and grid[r][c] == '.':
                    row_id[r][c] = row_count
                    c += 1
                row_count += 1
            else:
                c += 1

    # lo mismo por COLUMNAS
    col_id = [[-1] * n for _ in range(n)]
    col_count = 0
    for c in range(n):
        r = 0
        while r < n:
            if grid[r][c] == '.':
                while r < n and grid[r][c] == '.':
                    col_id[r][c] = col_count
                    r += 1
                col_count += 1
            else:
                r += 1

    # grafo bipartito: segmentos de fila (izq) --
    # segmentos de columna (der);
    # cada celda vacia es una arista posible entre ambos
    # segmentos
    adj = [[] for _ in range(row_count)]
    for r in range(n):
        for c in range(n):
            if grid[r][c] == '.':
                adj[row_id[r][c]].append(col_id[r][c])

main()
```

```
matched = [-1] * col_count
total = 0
for u in range(row_count):
    visited = [False] * col_count
    if try_kuhn(u, adj, matched, visited):
        total += 1
return total

def main():
    data = sys.stdin.read().split()
    idx = 0
    out = []
    while idx < len(data):
        n = int(data[idx]); idx += 1
        grid = data[idx:idx + n]; idx += n
        out.append(str(solve_board(n, grid)))
    sys.stdout.write("\n".join(out) + "\n")

main()
```

4.4 Componentes Fuertemente Conexas (Algoritmo de Kosaraju)

Preguntas guía — ¿cuándo usarlo?

- ¿Es un grafo dirigido y preguntan por "grupos" donde todo par de nodos es mutuamente alcanzable?
- ¿Necesito "condensar" el grafo (contraer cada componente en un nodo) para trabajar luego sobre un DAG?
- ¿n puede ser grande ($\sim 10^5$), por lo que hay que evitar recursión profunda en Python?

¿Para qué sirve? Particionar un grafo dirigido en componentes fuertemente conexas (SCC): subconjuntos maximales donde cada nodo puede llegar a cualquier otro del mismo subconjunto. Es la base para condensar grafos, resolver 2-SAT y detectar ciclos mutuos.

Complejidad: $O(V + E)$ tiempo y espacio (dos recorridos DFS).

Fundamento teórico: Kosaraju usa dos pasadas de DFS: (1) en el grafo original, registrar el orden de finalización (postorden) de cada nodo; (2) en el grafo transpuesto (aristas invertidas), recorrer los nodos en orden inverso de finalización. Cada árbol de DFS resultante en esta segunda pasada es exactamente una SCC, porque invertir las aristas separa los componentes que antes solo eran alcanzables en una dirección.

Ejercicio aplicado

Un juego tiene n planetas conectados por m teletransportadores dirigidos. Dos planetas pertenecen al mismo "reino" si hay ruta en ambos sentidos entre ellos. Determinar, para cada planeta, a qué reino pertenece.

E/S: $n \leq 10^5$ y $m \leq 2 \cdot 10^5 \rightarrow$ lectura total + tokenización (patrón 2). Además, el DFS se implementa de forma iterativa (con pila explícita) para evitar RecursionError en Python con grafos grandes.

Código solución

```
import sys

def kosaraju(n, adj):
    visited = [False] * n
    order = [] # orden de finalizacion (
    postorden)
```

```

for s in range(n):
    if visited[s]:
        continue
    visited[s] = True
    stack = [(s, iter(adj[s]))]          # iterador
    recuerda donde va en i parte de la lista
    while stack:
        node, it = stack[-1]
        advanced = False
        for nxt in it:                  # DFS iterativo
            con iteradores
            if not visited[nxt]:
                visited[nxt] = True
                stack.append((nxt, iter(adj[nxt])))
                advanced = True
                break
        if not advanced:                # no quedan
            vecinos: cierra el nodo
            order.append(node)
            stack.pop()

    radj = [[] for _ in range(n)]      # grafo
    transpuesto (aristas invertidas)
    for u in range(n):
        for v in adj[u]:
            radj[v].append(u)

    comp = [0] * n
    visited = [False] * n
    c = 0
    for node in reversed(order):        # orden inverso
        de finalizacion
        if visited[node]:
            continue
        c += 1
        visited[node] = True
        comp[node] = c
        stack = [node]
        while stack:
            u = stack.pop()
            for v in radj[u]:
                if not visited[v]:
                    visited[v] = True
                    comp[v] = c
                    stack.append(v)

    return c, comp

def solve():
    data = sys.stdin.buffer.read().split()
    idx = 0
    n = int(data[idx]); m = int(data[idx + 1]); idx += 2
    adj = [[] for _ in range(n + 1)]
    deg = [0] * (n + 1)
    for e in range(m):
        a = int(data[idx]); b = int(data[idx + 1]); idx += 2
        adj[a].append((b, e))
        adj[b].append((a, e))          # grafo no dirigido:
        ambos sentidos (e es id de arista)
        deg[a] += 1
        deg[b] += 1

    if deg[1] == 0 or any(deg[v] % 2 for v in range(1, n + 1)):
        print("IMPOSSIBLE")
        return

    used = [False] * m
    ptr = [0] * (n + 1)                # guarda las ids de
    las aristas
    path = []
    stack = [1]
    while stack:
        v = stack[-1]
        while ptr[v] < len(adj[v]) and used[adj[v][ptr[v]]]:
            ptr[v] += 1                # salta aristas
            ya usadas (O(1) amortizado)
        if ptr[v] < len(adj[v]):
            u, e = adj[v][ptr[v]]
            stack.append(u)
            used[e] = True
            ptr[v] += 1
    print(len(stack))

```

4.5 Circuito Euleriano (Algoritmo de Hierholzer)

Preguntas guía — ¿cuándo usarlo?

- ¿Piden recorrer TODAS las aristas exactamente una vez (no todos los nodos, como en Hamiltoniano)?
- ¿Se pide un camino o circuito que use cada calle/arista una sola vez?

- ¿Puedo verificar rápido la condición de existencia (grados pares para circuito; 0 ó 2 nodos de grado impar para camino) antes de construir la ruta?

¿Para qué sirve? Encontrar un recorrido que use cada arista del grafo exactamente una vez (circuito o camino euleriano). Aplicaciones: rutas de reparto/barrido que cubren todas las calles, o cualquier problema de "recorrer cada arista una vez".

Complejidad: $O(V + E)$ con la versión iterativa basada en pila (evita recursión y eliminaciones costosas de aristas).

Fundamento teórico: Existe circuito euleriano si y solo si el grafo es conexo (en aristas) y todos los vértices tienen grado par. El algoritmo de Hierholzer avanza por aristas no usadas hasta atascarse; en ese momento inserta el vértice atascado en la ruta final y retrocede. El resultado, leído en el orden en que se van "atascando" los vértices, es un circuito válido.

Ejercicio aplicado

Dado un mapa de n cruces y m calles bidireccionales, hallar una ruta que empiece y termine en el cruce 1 (oficina postal) y recorra cada calle exactamente una vez, o determinar que es "IMPOSSIBLE".

E/S: $n \leq 10^5$, $m \leq 2 \cdot 10^5 \rightarrow$ lectura total + tokenización (patrón 2). Se evita `adj.erase()/remove()` ($O(n)$ por operación, como en la versión C++ original) usando un arreglo de punteros `ptr[]` que salta aristas usadas: así la eliminación es $O(1)$ amortizado.

Código solución

```

import sys

def solve():
    data = sys.stdin.buffer.read().split()
    idx = 0
    n = int(data[idx]); m = int(data[idx + 1]); idx += 2

    adj = [[] for _ in range(n + 1)]
    deg = [0] * (n + 1)
    for e in range(m):
        a = int(data[idx]); b = int(data[idx + 1]); idx += 2
        adj[a].append((b, e))
        adj[b].append((a, e))          # grafo no dirigido:
        ambos sentidos (e es id de arista)
        deg[a] += 1
        deg[b] += 1

    if deg[1] == 0 or any(deg[v] % 2 for v in range(1, n + 1)):
        print("IMPOSSIBLE")
        return

    used = [False] * m
    ptr = [0] * (n + 1)                # guarda las ids de
    las aristas
    path = []
    stack = [1]
    while stack:
        v = stack[-1]
        while ptr[v] < len(adj[v]) and used[adj[v][ptr[v]]]:
            ptr[v] += 1                # salta aristas
            ya usadas (O(1) amortizado)
        if ptr[v] < len(adj[v]):
            u, e = adj[v][ptr[v]]
            stack.append(u)
            used[e] = True
            ptr[v] += 1
    print(len(stack))

```

```

    if ptr[v] == len(adj[v]):          # reviso todas
        las aristas de v y no queda ninguna disponible
        path.append(v)
        stack.pop()
    else:
        u, eid = adj[v][ptr[v]]
        used[eid] = True
        stack.append(u)                # avanza por la
        arista disponible

    if len(path) - 1 != m:              # no se
        recorrieron todas las aristas
        print("IMPOSSIBLE")
        return

    sys.stdout.write(" ".join(map(str, path)))

solve()

```

4.6 Puntos de Articulación (Algoritmo de Tarjan)

Preguntas guía — ¿cuándo usarlo?

- ¿Piden identificar nodos cuya eliminación desconecta el grafo (o aumenta el número de componentes)?
- ¿El grafo es no dirigido y se pregunta por robustez/conectividad ante fallos de un solo nodo?
- ¿n puede ser grande, por lo que hay que evitar recursión profunda en Python?

¿Para qué sirve? Encontrar los vértices de corte (articulation points) de un grafo no dirigido: nodos que, al eliminarse, aumentan el número de componentes conexas. Es la base para analizar robustez de redes (qué nodo, si falla, parte el sistema en pedazos).

Complejidad: $O(V + E)$ con DFS iterativo (un solo recorrido con low-link).

Fundamento teórico: Se hace un DFS calculando, para cada nodo, su tiempo de descubrimiento (disc) y el menor tiempo de descubrimiento alcanzable usando a lo más una arista de retroceso (low). Un nodo u (no raíz) es punto de articulación si tiene un hijo v en el árbol DFS tal que $\text{low}[v] \geq \text{disc}[u]$: eso significa que desde el subárbol de v no hay forma de "escapar" hacia arriba de u sin pasar por u. La raíz del DFS es un caso especial: es punto de articulación si tiene más de un hijo en el árbol.

Ejercicio aplicado

Dadas varias pruebas, cada una con n islas y m puentes bidireccionales (grafo conexo), contar cuántas islas, al hundirse, desconectarían partes de la ciudad. La entrada termina con un caso 0 0.

E/S: Múltiples casos con centinela "0 0" (patrón 3 de la sección de E/S). $n \leq 10^4$, $m \leq 10^5 \rightarrow$ lectura total + tokenización, y DFS iterativo con pila explícita (mismo motivo que en Kosaraju: evitar RecursionError/stack overflow).

Código solución

```

import sys

def find_articulation_points(n, adj):

```

```

disc = [0] * n
low = [0] * n
visited = [False] * n
is_ap = [False] * n
timer = 1

for start in range(n):
    if visited[start]:
        continue
    visited[start] = True
    disc[start] = low[start] = timer
    timer += 1
    root_children = 0
    # pila: [nodo, padre, iterador de vecinos,
    # ya_salto_arista_al_padre]
    stack = [[start, -1, iter(adj[start]), False]]
    while stack:
        frame = stack[-1]
        node, parent, it, _ = frame
        advanced = False
        for nxt in it:
            if nxt == parent and not frame[3]:
                frame[3] = True # salta SOLO una
                vez la arista de vuelta
                continue # (por si hay
                aristas paralelas)
            if not visited[nxt]:
                visited[nxt] = True
                disc[nxt] = low[nxt] = timer
                timer += 1
                if node == start:
                    root_children += 1
                stack.append([nxt, node, iter(adj[nxt]), False])
                advanced = True
                break
            else:
                low[node] = min(low[node], disc[nxt])
        # arista de retroceso
        if not advanced:
            stack.pop()
        if stack:
            par = stack[-1]
            par_node = par[0]
            low[par_node] = min(low[par_node], low[node])
        # condicion clasica de punto de
        # articulacion (no raiz)
        if par_node != start and low[node] >= disc[par_node]:
            is_ap[par_node] = True
        if root_children > 1: # caso especial: la
            raiz del DFS
            is_ap[start] = True

    return is_ap

def main():
    data = sys.stdin.buffer.read().split()
    idx = 0
    out = []
    while True:
        n = int(data[idx]); m = int(data[idx + 1]); idx += 2
        if n == 0 and m == 0: # centinela de fin
            break
        adj = [[] for _ in range(n)]
        for _ in range(m):
            u = int(data[idx]) - 1; v = int(data[idx + 1]) - 1; idx += 2
            adj[u].append(v)
            adj[v].append(u)
        is_ap = find_articulation_points(n, adj)
        out.append(str(sum(is_ap)))

```

```
sys.stdout.write("\n".join(out) + "\n")

main()
```

5 Búsqueda y Optimización

5.1 Búsqueda Binaria sobre la Respuesta + Two Pointers

Preguntas guía — ¿cuándo usarlo?

- ¿La respuesta es un número y existe una condición monótona (si X funciona, todo valor mayor/menor también funciona)?
- ¿Puedo verificar en $O(n)$ o $O(n \log n)$ si un valor candidato "cumple" la condición pedida?
- ¿Dentro de esa verificación necesito contar pares/elementos con una condición de suma sobre un arreglo ordenado? (ahí entra Two Pointers)

¿Para qué sirve? Cuando no es fácil calcular la respuesta directamente pero sí verificar si un candidato es válido, y esa validez es monótona, se "busca" la respuesta con búsqueda binaria sobre el rango de valores posibles. Two Pointers acelera el conteo en arreglos ordenados de $O(n^2)$ a $O(n)$.

Complejidad: Búsqueda binaria: $O(\log(\text{rango}))$ iteraciones. Cada verificación con Two Pointers: $O(n)$. Total: $O(n \log(\text{rango}))$.

Fundamento teórico: La búsqueda binaria sobre la respuesta funciona porque el conjunto de valores "válidos" forma un intervalo continuo (monotonía), reduciendo el espacio de búsqueda a la mitad en cada paso. Two Pointers explota que, en un arreglo ordenado, mover un extremo cambia la condición de suma de forma predecible, evitando recorrer todas las parejas.

Ejercicio aplicado

Dados N precios de categorías de joyas, un cliente que es el K -ésimo en la fila debe comprar dos joyas de categorías distintas sin repetir una combinación ya usada por otro cliente. Hallar el mínimo dinero que garantiza poder comprar: la K -ésima suma más pequeña entre todos los pares de categorías distintas.

E/S: N no está acotado explícitamente pero puede ser grande $\rightarrow$ lectura total + tokenización (patrón 2) para el arreglo de precios.

Código solución

```
import sys

def count_pairs_leq(prices, n, limit):
    # cuenta cuantos pares cumplen arr[i] + arr[j] <= limit
    # usando dos punteros sobre un arreglo YA ORDENADO
    count = 0
    left, right = 0, n - 1
    while left < right:
        if prices[left] + prices[right] <= limit:
            count += right - left  # todos los pares
            con "left" fijo
            left += 1
        else:
```

```
        right -= 1
    return count

def solve():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); k = int(data[1])
    prices = list(map(int, data[2:2 + n]))
    prices.sort()

    low = prices[0] + prices[1]  # menor suma posible
    high = prices[n - 2] + prices[n - 1]  # mayor suma posible

    while low < high:
        mid = low + (high - low) // 2
        if count_pairs_leq(prices, n, mid) >= k:
            high = mid  # mid cumple: intenta algo menor
        else:
            low = mid + 1  # mid no cumple: sube el limite

    print(low)

solve()
```

5.2 —NEED: Enumeración por Más-caras de Bits (Bitmask Brute Force)

Preguntas guía — ¿cuándo usarlo?

- ¿Hay un pequeño número ($\leq \sim 20$) de decisiones binarias desconocidas (bits, elecciones sí/no) y hace falta probar combinaciones?
- ¿El espacio total de combinaciones (2^k) es lo bastante chico para probarlas todas dentro del límite de tiempo?
- ¿Cada combinación se puede armar y evaluar rápido a partir de su representación binaria?

¿Para qué sirve? Representar y recorrer exhaustivamente todas las configuraciones posibles de un conjunto pequeño de decisiones binarias. Cada máscara de bits codifica una asignación distinta de valores, permitiendo generar candidatos de forma sistemática y verificar cuál satisface la condición del problema.

Complejidad: $O(2^k \cdot \text{costo de evaluar una combinación})$, con k = número de incógnitas binarias.

Fundamento teórico: Cualquier asignación de k bits desconocidos corresponde biunívocamente a un entero entre 0 y $2^k - 1$. Recorriendo ese rango y extrayendo el bit j con $(\text{mascara} \gg j) \& 1$, se reconstruye cada combinación posible sin necesidad de recursión ni backtracking explícito. Es la enumeración más directa de un espacio de búsqueda combinatorio pequeño.

Ejercicio aplicado

Un mensaje M y un divisor N se recibieron con hasta 16 bits en total marcados como perdidos (*). Reconstruir un mensaje M compatible (M completando los '*' de M) tal que exista un N (completando los '*' de N) donde N divide a M exactamente.

E/S: $|M'| \leq 500$ pero el total de '*' (en M' y N' combinados) es $\leq 16 \rightarrow$ lectura total con readline es más que suficiente; el costo real está en las 2^{16} combinaciones, no en la entrada.

Código solución

```
import sys

input = sys.stdin.readline

def generate(strings):
    pos = [] # posiciones donde existen bits desconocidos
    for i, c in enumerate(strings):
        if c == '*':
            pos.append(i)
    k = len(pos)

    ans = []
    for mask in range(1 << k): # cada mascara representa una asignacion distinta de los k bits perdidos
        cur = strings[:]
        for j in range(k):
            if (mask >> j) & 1:
                cur[pos[j]] = '1'
            else:
                cur[pos[j]] = '0'
        ans.append(''.join(cur))
    return ans

M = list(input().strip())
N = list(input().strip())

messages = generate(M)
divisors = generate(N)

for m in messages: # probar todas las combinaciones posibles
    value_m = int(m, 2)
    for n in divisors:
        value_n = int(n, 2)

        if value_n == 0:
            continue # la division entre cero no existe
        if value_m % value_n == 0: # la primera combinacion valida es respuesta
            print(m)
            sys.exit()
```

6 Matemáticas y Simulación

6.1 Patrón de Reducción Binaria (tipo Josefo)

Preguntas guía — ¿cuándo usarlo?

- ¿El problema describe un proceso repetitivo/alternado (cada segundo elemento se "marca") sobre n elementos, con n muy grande ($\geq 10^9$)?
- ¿Simular paso a paso sería $O(n)$, demasiado lento, pero el patrón se repite de forma autosimilar al dividir por la mitad?
- ¿Puedo replantear el problema de tamaño n como un subproblema de tamaño $\sim n/2$ aplicando la misma regla?

¿Para qué sirve? Resolver en $O(\log n)$ problemas de simulación con patrones alternados sobre secuencias enormes, sin simular cada paso. Es la misma idea del clásico Problema de Josefo: en vez de eliminar uno por uno, se "salta" de una ronda completa a la mitad del problema restante.

Complejidad: $O(\log n)$ por consulta (cada ronda aprox. divide n a la mitad).

Fundamento teórico: En cada "ronda" del proceso, la mitad de los elementos son descartados/activados según la paridad de su posición. Reformulando la posición y el tamaño del problema tras cada ronda ($n' = n -$ procesados, i' = nueva posición relativa) se llega a una recurrencia que converge en $O(\log n)$ pasos en vez de $O(n)$.

Ejercicio aplicado

Hay N personas en fila y un detector de metales defectuoso que se activa de forma alternada (la 1ª persona no lo activa, la 2ª sí, la 3ª no, ...; quien lo activa va al final de la fila). Dado que el coach está en la posición i , determinar en qué turno (entre quienes no activan el detector) pasará.

E/S: $T \leq 1000$ casos, cada uno con solo dos enteros $\rightarrow$ readline por línea (patrón 1) es suficiente y más simple.

Código solución

```
import sys
input = sys.stdin.buffer.readline

def safe_position(n, i):
    # turno "seguro" (que no activa el patron) en que pasara la posicion i
    res = 0
    while i % 2 == 0: # mientras i cae en la mitad "activada"
        res += i // 2 # cuenta los turnos seguros de esta ronda
        n -= i // 2 # reduce el problema a la mitad restante
        i = n # la posicion i se convierte en el nuevo n
    res += (i + 1) // 2 # suma los turnos seguros de la ronda final
    return res

def solve():
    t = int(input())
    out = []
    for _ in range(t):
        n, i = map(int, input().split())
        out.append(str(safe_position(n, i)))
    sys.stdout.write("\n".join(out))

solve()
```

6.2 Fuerza Bruta / Simulación Directa

Preguntas guía — ¿cuándo usarlo?

- ¿El tamaño de la entrada es pequeño ($N \leq 10^3$ o similar) y no hay una fórmula cerrada evidente?
- ¿Puedo calcular la respuesta iterando directamente sobre el espacio del problema sin que se dispare el tiempo?
- ¿El problema es más de matemáticas/probabilidad que de estructuras de datos?

¿Para qué sirve? Reconocer cuándo NO hace falta un algoritmo sofisticado: con restricciones pequeñas, iterar directamente (simulando la definición del problema) es la solución más simple, rápida de programar y menos propensa a errores. Identificar esto ahorra tiempo valioso en el contrarreloj.

Complejidad: Depende del problema; en este caso $O(n)$ con aritmética de punto flotante.

Fundamento teórico: Se calcula directamente la probabilidad de que NO haya coincidencia de cumpleaños entre k personas, multiplicando factores $(1 - (k-1)/n)$ sucesivamente (extensión del problema del cumpleaños), deteniéndose apenas esa probabilidad cae por debajo de 0.5 (la probabilidad de coincidencia supera el 50%).

Ejercicio aplicado

Dado N (número de días del año), hallar el mínimo número de personas necesarias en una sala para que la probabilidad de que dos compartan cumpleaños supere el 50%.

E/S: Entrada de un solo entero $\rightarrow$ readline (patrón 1), no requiere nada especial.

Código solución

```
import sys
input = sys.stdin.buffer.readline

n = int(input())
prob_no_coincidencia = 1.0
for k in range(1, n + 2):
    prob_no_coincidencia *= (n - (k - 1)) / n
    if prob_no_coincidencia < 0.5:
        print(k)
        break
```

6.3 Simulación Lineal (Máximo de Prefijo)

Preguntas guía — ¿cuándo usarlo?

- ¿La respuesta depende de un solo recorrido lineal del arreglo, calculando un máximo/mínimo/acumulado por posición?
- ¿No hay dependencias entre posiciones más allá de una comparación directa fila por fila?
- ¿El problema, aunque tenga una narrativa elaborada, se reduce matemáticamente a una fórmula simple tras identificar la variable clave?

¿Para qué sirve? Resolver directamente problemas donde, tras modelar bien la situación (aquí: el "cuello de botella" es la fila con menor espacio libre entre dos paredes), un solo recorrido $O(n)$ calculando un máximo/mínimo basta. Es clave en competencia reconocer cuándo un problema de apariencia compleja se reduce a esto.

Complejidad: $O(h)$ tiempo, $O(h)$ espacio (o $O(1)$ extra si se procesa en streaming).

Fundamento teórico: Las dos paredes se mueven a 1 m/s cada una. En la fila i , el espacio libre es $w - (l[i] + r[i])$ y se cierra a razón de 2 m/s (ambas paredes se acercan). El tiempo en que la fila i se cierra es $(w - l[i] - r[i]) / 2$.

Las paredes chocan en la fila que se cierra primero: la de menor espacio libre inicial, equivalente al máximo de $(l[i] + r[i])$.

Ejercicio aplicado

Una sala de H filas y ancho W tiene una pared este y una pared oeste que se acercan a 1 m/s cada una desde que alguien entra. Cada fila i tiene un ancho ya ocupado por cada pared. Determinar en cuánto tiempo chocan las paredes.

E/S: $H \leq 10^5$, dos arreglos de tamaño $H \rightarrow$ lectura total + tokenización (patrón 2).

Código solución

```
import sys

def solve():
    data = sys.stdin.buffer.read().split()
    h = int(data[0]); w = int(data[1])
    left = list(map(int, data[2:2 + h]))
    right = list(map(int, data[2 + h:2 + 2 * h]))
    max_sum = max(left[i] + right[i] for i in range(h))
    ans = (w - max_sum) / 2

if ans.is_integer():
    # verifica parseo de float a
    int
    print(int(ans))
else:
    print(ans)
solve()
```

6.4 +Ordenamiento de Puntos Críticos y Búsqueda de Huecos—

Preguntas guía — ¿cuándo usarlo?

- ¿El problema depende de encontrar el mayor (o menor) espacio libre entre eventos/puntos dispersos en una línea de tiempo o recta?
- ¿Los puntos relevantes (incluyendo extremos del rango) se pueden ordenar y luego recorrer una sola vez?
- ¿La respuesta se reduce a comparar diferencias entre elementos consecutivos ya ordenados?

¿Para qué sirve? Determinar si existe un hueco suficientemente grande entre puntos dispersos (tiempos, posiciones) en un intervalo, agregando los extremos del intervalo como puntos artificiales y ordenando todo junto. Evita simular minuto a minuto: basta con mirar diferencias consecutivas tras ordenar.

Complejidad: $O(n \log n)$ por el ordenamiento; $O(n)$ el resto.

Fundamento teórico: Cualquier hueco maximal sin eventos queda delimitado por dos puntos consecutivos en el arreglo ordenado (incluyendo los extremos del intervalo total como puntos ficticios). Por lo tanto, el hueco más grande siempre es la mayor diferencia entre elementos consecutivos de la lista ordenada; no hace falta considerar ninguna otra combinación de puntos.

Ejercicio aplicado

Un vuelo dura D minutos y sirve M comidas en instantes dados; hace falta dormir T minutos seguidos sin interrupción. Determinar si es posible dormir T minutos consecutivos sin perderse ninguna comida.

E/S: $M \leq 1000 \rightarrow$ entrada pequeña, readline por línea (patrón 1) es suficiente.

Código solución

```
import sys
input = sys.stdin.buffer.readline

def solve():
    T, D, M = map(int, input().split())
    times = [0, D] # extremos del
    # vuelo como puntos ficticios
    for _ in range(M):
        y = int(input())
        times.append(y)
    times.sort()

    ans = 'N'
    for i in range(1, len(times)):
        if times[i] - times[i - 1] >= T: # hueco
            # suficiente para dormir T minutos
            ans = 'Y'
            break
    print(ans)

solve()
```

6.5 +Teoría de Números: Regla de Divisibilidad Alternada—

Preguntas guía — ¿cuándo usarlo?

- ¿El problema pide verificar o forzar divisibilidad por $(base \pm 1)$ de un número representado por dígitos?
- ¿Se puede aprovechar que $base \equiv -1 \pmod{base+1}$, lo que convierte la suma ponderada de dígitos en una suma ALTERNADA módulo $(base+1)$?
- ¿Se pide además construir/ajustar un dígito específico para lograr esa divisibilidad?

¿Para qué sirve? Generalizar la clásica regla de divisibilidad por 11 en base 10 (suma alterna de dígitos) a cualquier base B , para verificar o forzar divisibilidad por $B+1$ sin operar con el número completo (que puede tener hasta $2 \cdot 10^5$ dígitos).

Complejidad: $O(L)$, con L = cantidad de dígitos.

Fundamento teórico: Como $B \equiv -1 \pmod{B+1}$, cada potencia $B^i \equiv (-1)^i \pmod{B+1}$. Por lo tanto, el valor del número módulo $B+1$ es exactamente la suma alternada de sus dígitos (sumando los de posición par, restando los de posición impar). Para forzar que esa suma sea 0 módulo $B+1$ cambiando un solo dígito D_i por un valor MENOR, se despeja cuánto debe valer ese dígito para anular el residuo actual, y se verifica que el nuevo valor sea válido (entre 0 y el dígito original, sin aumentarlo).

Ejercicio aplicado

Dado un número N de L dígitos en base B , encontrar un dígito que se pueda reducir (nunca aumentar) para que el número resultante sea múltiplo de $B+1$, priorizando

el menor valor final posible, o determinar que no existe solución.

E/S: $L \leq 2 \cdot 10^5 \rightarrow$ aunque el código usa `input()` estándar por simplicidad (ya venía así y funciona), para esta escala también sería válido leer con `sys.stdin.buffer.read().split()` (patrón 2) si se buscara el máximo rendimiento.

Código solución

```
# Lee la base B y la cantidad de dígitos L
basedig = list(map(int, input().split()))
nums = list(map(int, input().split())) # dígitos D_1..
# D_L, de mas a menos significativo

base = basedig[0]
div = basedig[0] + 1 # queremos que N
# sea múltiplo de base+1
dig = basedig[1] # L = cantidad
# de dígitos

# suma alternada de dígitos == N mod (base+1), porque
# base = -1 mod (base+1)
n = 0
for i in range(len(nums)):
    if (dig - 1 - i) % 2 == 0:
        n += nums[i]
    else:
        n -= nums[i]
n %= div

if n == 0:
    print("0 0") # ya es múltiplo
    # no hace falta cambiar nada
    exit()

# buscar el primer dígito (de izquierda a derecha) que se
# pueda REDUCIR
# para anular el residuo n, segun la paridad de su
# posicion
res = 0
for i in range(len(nums)):
    if (dig - 1 - i) % 2 == 0:
        res = n # esta posicion
        # suma: hay que restarle 'n'
    else:
        res = div - n # esta posicion
        # resta: se compensa distinto
    if 1 <= res <= nums[i]: # solo vale si
        # es una reduccion real (no negativa)
        print(f"{i + 1} {nums[i] - res}")
        exit()

print("-1 -1") # ningun digito
# puede arreglarlo
```

6.6 +Simulación de Eventos con Colas—

Preguntas guía — ¿cuándo usarlo?

- ¿Hay un recurso compartido con estado (dirección, disponibilidad) que cambia según el orden de llegada de eventos con timestamps?
- ¿Los eventos ya vienen ordenados (o se pueden ordenar) y conviene procesarlos con colas separadas por tipo?
- ¿El estado siguiente depende de comparar el próximo evento contra un tiempo límite (fin de uso actual del recurso)?

¿Para qué sirve? Simular un sistema con estado (aquí: una dirección de movimiento y un instante de fin de uso) que evoluciona a medida que llegan eventos de distintos tipos, usando una cola por tipo de evento en vez de simular minuto a minuto.

Complejidad: $O(n)$ — cada persona se encola y desencola una sola vez.

Fundamento teórico: Se mantienen dos colas (una por dirección deseada) y un estado (dirección activa, instante en que el recurso queda libre). En cada paso se mira el evento más próximo entre ambas colas: si coincide con la dirección activa, extiende el fin de uso; si es la dirección contraria, esa persona debe esperar (se cuenta pero no se procesa aún); si ya no hay eventos antes de que el recurso quede libre, se decide si invertir la dirección (si había gente esperando) o terminar ese ciclo de uso.

Ejercicio aplicado

N personas llegan a una escalera doble (10 segundos por cruce) en tiempos distintos, cada una queriendo ir en una de dos direcciones. Determinar el instante en que la última persona termina de usarla.

E/S: $n \leq 10^4 \rightarrow$ lectura total + tokenización (patrón 2); se usan dos deque (collections.deque) en vez de queue, ya que en Python deque tiene $O(1)$ real para popleft().

Código solución

```
import sys
from collections import deque

def solve():
    data = sys.stdin.buffer.read().split()
    idx = 0
    n = int(data[idx]); idx += 1
    left = deque() # personas que quieren ir en
    direccion 0
    right = deque() # personas que quieren ir en
    direccion 1
    for _ in range(n):
        t = int(data[idx]); d = int(data[idx + 1]); idx
        += 2
        (left if d == 0 else right).append(t)

    end = 0 # instante en que el recurso queda
    libre
    ans = 0
    waiting = 0 # cuantos esperan en direccion
    contraria a la activa
    direction = -1 # -1 = detenida

    while left or right or direction != -1:
        # mira el proximo evento disponible entre ambas
        # colas
        next_time, who = None, -1
        if left and (next_time is None or left[0] <
        next_time):
            next_time, who = left[0], 0
        if right and (next_time is None or right[0] <
        next_time):
            next_time, who = right[0], 1

        if direction == -1:
            # arranca detenida: la toma quien llegue
            primero
            direction = who
            end = next_time + 10
            (left if who == 0 else right).popleft()

        elif who != -1 and next_time < end:
            if who == direction:
                end = max(end, next_time + 10) # se
                sube mientras sigue en marcha
            else:
                waiting += 1 #
                direccion contraria: debe esperar
                (left if who == 0 else right).popleft()
            else:
                # no hay mas eventos antes de que el recurso
                quede libre
                if waiting > 0:
                    direction = 1 - direction # invierte
                    sentido para atender a los que esperan
                    end += 10
                    waiting = 0
                else:
                    ans = end
                    direction = -1 # se detiene

    print(ans)

solve()
```

```
elif who != -1 and next_time < end:
    if who == direction:
        end = max(end, next_time + 10) # se
        sube mientras sigue en marcha
    else:
        waiting += 1 #
        direccion contraria: debe esperar
        (left if who == 0 else right).popleft()
    else:
        # no hay mas eventos antes de que el recurso
        quede libre
        if waiting > 0:
            direction = 1 - direction # invierte
            sentido para atender a los que esperan
            end += 10
            waiting = 0
        else:
            ans = end
            direction = -1 # se detiene

print(ans)

solve()
```

7 Construcción y Combinatoria

7.1 Algoritmos Constructivos: Invariantes y Paridad

Preguntas guía — ¿cuándo usarlo?

- ¿El problema pide construir explícitamente una configuración (o probar que es imposible), en vez de optimizar o contar?
- ¿Puedo idear una construcción simple (simetrías, intercambios, patrones repetidos) en vez de buscar entre todas las opciones?
- ¿La imposibilidad de otros valores se puede demostrar con un invariante (paridad, módulo, suma fija) que ninguna elección puede romper?

¿Para qué sirve? Resolver problemas donde no hace falta ningún algoritmo de búsqueda: basta con proponer una construcción explícita y justificar, con un argumento de invariante, que el resultado contrario (o cualquier otro valor puntual) nunca puede ocurrir sin importar qué elecciones se hagan.

Complejidad: $O(n)$ por caso — construir y emitir la respuesta.

Fundamento teórico: Se particiona $\{1, \dots, n\}$ en pares consecutivos $(2k-1, 2k)$ y se intercambian entre sí. Cada par contribuye al contador un valor relacionado con $V_k = (2k-1)(2k)$, que siempre es par; y ese aporte entra a la suma final como $-V_k$, 0 o V_k (las tres opciones son pares). Como la suma de términos pares es par, el resultado nunca puede ser impar — en particular, nunca puede ser 1.

Ejercicio aplicado

(Codeforces 2246A — farmpiggy and Subset Sum)
Dado un entero par n , construir una permutación p de longitud n tal que, sin importar qué signo (sumar, restar o ignorar) se le asigne a cada término $i \cdot p_i$ al acumular un

contador, el resultado final nunca pueda ser exactamente 1.

E/S: $t \leq 25$, $n \leq 50$ (par) $\rightarrow$ entrada pequeña, readline por línea (patrón 1) es suficiente.

Código solución

```
import sys
input = sys.stdin.buffer.readline

t = int(input())
out = []
for _ in range(t):
    n = int(input())
    p = []
    for i in range(1, n + 1):
        # empareja (1,2), (3,4), ..., (n-1,n) e
        # intercambia cada pareja:
        # el aporte de cada pareja al contador siempre es
        # par (ver teoria)
        if i % 2 == 1:
            p.append(i + 1)      # posicion impar ->
            # apunta a la siguiente
        else:
            p.append(i - 1)      # posicion par -> apunta
            # a la anterior
    out.append(' '.join(map(str, p)))
sys.stdout.write('\n'.join(out) + '\n')
```

7.2 Algoritmos Constructivos: Construcción Explícita

Preguntas guía — ¿cuándo usarlo?

- ¿Piden construir explícitamente n valores con una relación de divisibilidad o proporción exacta entre ellos y su suma?
- ¿La condición se puede reescribir como una ecuación entre fracciones que deben sumar exactamente 1?
- ¿Existe una descomposición numérica conocida (como una fracción egipcia) que se pueda extender o escalar para cualquier n ?

¿Para qué sirve? Construir directamente arreglos o valores que cumplan una condición de divisibilidad o proporción exacta, usando descomposiciones numéricas conocidas en vez de búsquedas o heurísticas. Es frecuente en problemas "constructivos" donde la solución es una fórmula cerrada, no un algoritmo de recorrido.

Complejidad: $O(n)$ por caso.

Fundamento teórico: Si cada $a_i = S / k_i$ (S = suma total), la condición " S divisible por cada a_i " equivale a que los k_i sean enteros positivos distintos con $\sum(1/k_i) = 1$. Partiendo de la identidad $1 = 1/2 + 1/3 + 1/6$ y extendiendo el término $1/6$ con una cadena de potencias de 2, se obtiene una familia de k_i distintos para cualquier $n \geq 3$: la construcción cerrada $\{1, 2, 3, 6, 12, 24, \dots, 3 \cdot 2^{(n-3)}\}$. Los casos $n=1$ y $n=2$ se resuelven aparte (trivial e imposible, respectivamente).

Ejercicio aplicado

(Codeforces 2246B — ezraft and Array) Dado n , construir un arreglo de n enteros positivos distintos tal que la suma total sea divisible por cada uno de sus elementos, o determinar que no es posible.

E/S: $t \leq 50$, $n \leq 50 \rightarrow$ entrada pequeña, readline por línea (patrón 1).

Código solución

```
import sys
input = sys.stdin.buffer.readline

t = int(input())
out = []
for _ in range(t):
    n = int(input())
    if n == 1:
        out.append('1')      # caso base: un
        # unico elemento
    elif n == 2:
        out.append('-1')      # a1/a2 y a2/a1
        # con ambos positivos
        # obliga a1 ==
        # a2 (no son distintos)
    else:
        # construccion cerrada a partir de  $1 = 1/2 + 1/3 + 1/6$ :
        # {1, 2} mas una cadena de potencias de 2
        # escaladas por 3,
        # que nunca "chocan" con los divisores de 1 o 2
        vals = [1, 2] + [3 * (1 << k) for k in range(n - 2)]
        out.append(' '.join(map(str, vals)))
sys.stdout.write('\n'.join(out) + '\n')
```

7.3 Combinatoria: Conteo por Simetría de Binomios

Preguntas guía — ¿cuándo usarlo?

- ¿Piden contar subconjuntos o subsecuencias que cumplen una condición de suma o balance?
- ¿Hay valores repetidos en la entrada, de forma que agruparlos por valor simplifique el conteo?
- ¿Se puede usar $C(c, \text{par}) = C(c, \text{impar}) = 2^{(c-1)}$ para separar el conteo por grupo y combinar con una potencia de 2?

¿Para qué sirve? Contar eficientemente subconjuntos o subsecuencias que satisfacen una condición combinatoria, evitando enumerar los 2^n subconjuntos, aprovechando la simetría de los coeficientes binomiales para reducir el problema a una subsecuencia mucho más pequeña: solo los valores distintos.

Complejidad: $O(n)$ por caso (la entrada ya viene no-decreciente).

Fundamento teórico: Dentro de un grupo de c copias iguales, elegir una cantidad par de ellas no cambia la suma alternada acumulada (aporta 0) y elegir una cantidad impar sí la cambia (aporta $\pm \text{valor}$). Como $C(c, \text{par}) = C(c, \text{impar}) = 2^{(c-1)}$, cada grupo aporta el mismo peso sin importar el modo elegido, así que el problema se reduce a contar subconjuntos de suma alternada 0 sobre los valores DISTINTOS, multiplicando el resultado por $2^{(n-m)}$ al final (m = cantidad de valores distintos). En esa secuencia reducida, agrupando términos de dos en dos se demuestra que la única forma de obtener alternada 0 con más de un elemento es usando exactamente 3 valores: -1 y un par de enteros positivos consecutivos.

Ejercicio aplicado

(Codeforces 2246C — Omar and Alternating Sums)

Dado un arreglo no decreciente de n valores (cada uno -1 o un entero positivo), contar cuántas subsecuencias tienen suma alternada (primer término positivo, segundo negativo, tercero positivo, ...) igual a 0, módulo 10^9+7 .

E/S: $t \leq 10^4$, $\Sigma n \leq 2 \cdot 10^5 \rightarrow$ volumen moderado; se usa readline por línea (patrón 1), adecuado porque el costo dominante está en el procesamiento $O(n)$, no en la lectura.

Código solución

```
import sys
input = sys.stdin.buffer.readline

MOD = 10**9 + 7

t = int(input())
out = []
for _ in range(t):
    n = int(input())
    a = list(map(int, input().split()))

    # el arreglo ya viene no-decreciente: colapsar a
    # valores distintos.
    # cada grupo de c repetidos aporta un factor 2^(c-1)
    # independiente
    # del resto (C(c,par) == C(c,impar) == 2^(c-1))
    distinct = []
    for x in a:
        if not distinct or distinct[-1] != x:
            distinct.append(x)

    m = len(distinct)
    has_neg1 = 1 if distinct[0] == -1 else 0 # -1 es el
    # unico valor <= 0 posible

    pos = [v for v in distinct if v > 0]
    count_consec = 0
    for i in range(len(pos) - 1):
        if pos[i + 1] - pos[i] == 1: # pares
            # de enteros positivos consecutivos
            count_consec += 1

    # en la secuencia reducida, la suma alternada solo
    # puede dar 0 con:
    # el subconjunto vacio, o exactamente {-1, v, v+1}
    # para cada pareja
    # consecutiva (si hay un -1 disponible)
    reduced = (1 + (count_consec if has_neg1 else 0)) %
    MOD
    ans = reduced * pow(2, n - m, MOD) % MOD #
    # reincorpora los repetidos
    out.append(str(ans))

sys.stdout.write('\n'.join(out) + '\n')
```

7.4 +Verificación de Factibilidad por Invariante de Clases—

Preguntas guía — ¿cuándo usarlo?

- ¿Los elementos están agrupados en clases (colores, tipos) y solo se permiten intercambios DENTRO de la misma clase?
- ¿Se pregunta si una configuración objetivo es alcanzable, en vez de contarlas o construirlas?

- ¿La clase de cada posición es invariante (no cambia con los intercambios permitidos), y solo importa qué elemento termina en cada posición?

¿Para qué sirve? Verificar si una configuración objetivo es alcanzable cuando las únicas operaciones permitidas son intercambios dentro de una misma clase (color, grupo). Como los intercambios dentro de una clase permiten CUALQUIER permutación entre sus posiciones, el problema se reduce a comparar, para cada posición, si la clase requerida coincide con la clase disponible.

Complejidad: $O(n \log n)$ por el ordenamiento.

Fundamento teórico: Intercambiar repetidamente elementos de una misma clase genera el grupo simétrico completo sobre las posiciones de esa clase: cualquier permutación interna es alcanzable. Por lo tanto, la clase (color) de cada posición es un invariante que ningún intercambio permitido puede romper. La configuración ordenada es alcanzable si y solo si, al ordenar los elementos por su valor objetivo, la clase que le corresponde a cada uno coincide con la clase original de esa posición.

Ejercicio aplicado

Una secuencia de n bloques, cada uno con un número distinto (1..n) y un color, se puede reordenar intercambiando bloques del mismo color. Determinar si es posible dejar la secuencia ordenada ascendentemente por número.

E/S: $n \leq 10^5 \rightarrow$ lectura línea a línea (patrón 1) es aceptable dado que el costo dominante es el ordenamiento, no la lectura.

Código solución

```
import sys
input = sys.stdin.buffer.readline

def solve():
    n, k = map(int, input().split())
    color_at_position = [0] * n # color de cada
    # posicion (invariante)
    blocks = [] # (numero, color) de
    # cada bloque
    for i in range(n):
        num, c = map(int, input().split())
        color_at_position[i] = c
        blocks.append((num, c))

    blocks.sort() # ordena por numero:
    # donde deberia quedar cada uno
    # si el color del bloque que llega a la posicion i
    # coincide con el color
    # que YA tenia esa posicion, es alcanzable (el color
    # nunca cambia de lugar)
    ok = all(blocks[i][1] == color_at_position[i] for i
    in range(n))
    print('Y' if ok else 'N')

solve()
```